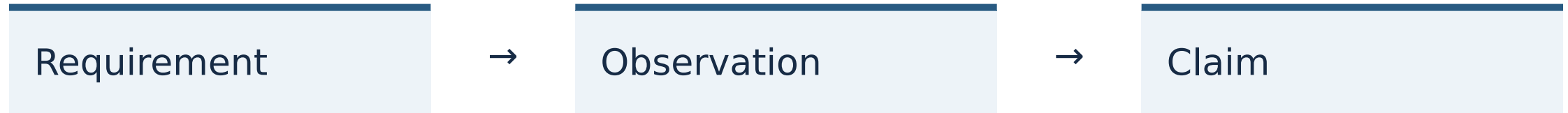


Foundations of hardware verification

A passing test is evidence with a defined scope.



Learning outcomes

Make a verification claim that another engineer can audit.

- Distinguish verification, validation and manufacturing test.
- Identify shared assumptions between design and checker.
- Select an oracle and state its limits.
- Demand evidence for a stage exit.

Route through the lesson

Four questions connect the foundations.

- Why do escapes survive?
- Where can independent checking fail?
- What can a test actually establish?
- When is a block ready for its next use?

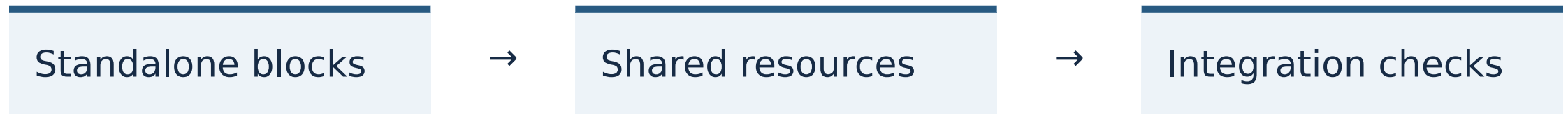
Scope before method

Different checks answer different questions.



A bug escape crosses a boundary

Block correctness does not automatically compose.



- Correct blocks can interact incorrectly.
- Arbitration, ordering and back-pressure emerge during integration.

Read the denominator

Survey headlines can measure different outcomes.

- First-silicon success concerns IC/ASIC fabrication.
- FPGA field escapes concern shipped systems.
- Scope and uncertainty belong beside the percentage.

Discussion: move verification to the lab?

A cheap reprogramming step does not make an escape cheap.

- An FPGA schedule cuts pre-lab verification.
- Which new behavior still needs model-level checking?
- What observability would the team lose?

Independence starts at interpretation

Separate implementations can share the same wrong assumption.



- A checker copied from the RTL can reproduce its mistake.
- Independent interpretation must reach the comparison.

Find the reconvergence point

Shared machinery can remove the independence a test needs.

- Trace what the design and checker reuse.
- Ask which defect that reuse would hide.

Stress the contract

A verifier should expose violations instead of repairing them.



- Exercise legal corner conditions.
- Keep the observer strict.
- Do not normalize a faulty response into an acceptable one.

Discussion: whose expected value?

An agreement can be a common-mode failure.

- The scoreboard uses the designer's address helper.
- All comparisons pass.
- Which failure could remain invisible?

The space exceeds enumeration

Configuration, internal state and history all matter.



- An abstract model removes detail; it does not prove omitted behavior irrelevant.

Place checks where failures are observable

Controllability and observability determine useful verification levels.

- A feature must be provoked and its failure seen.
- Moving a check upward can destroy either capability.

Use complementary oracles

Protocol legality does not establish payload correctness.



- Assertions check bounded statements.
- A scoreboard checks predicted outcomes.
- Completeness checks catch missing transactions.

Discussion: a clean run with no comparisons

Activity and correctness require different evidence.

- The regression reports no failure.
- No output transaction was compared.
- What claim can the report support?

Plan the evidence before implementation

A test list is not a closure argument.

- Name the feature and its checker.
- Define closure evidence and ownership.
- Review assumptions before building the environment.

Bring-up is a checked transaction

Traffic without an active oracle is not verification bring-up.



- Demonstrate stimulus, observation and checking together.
- Then grow regression and coverage maturity.

Maturity is demonstrated

Stage labels summarize checkable conditions.

- Implemented stimulus and checkers.
- Coverage and sustained regression evidence.
- Reviewed exclusions and open-bug dispositions.

Discussion: ready for integration?

The consumer needs a defensible entry criterion.

- The block generates traffic but its checker is unfinished.
- Another team wants to build on it.
- Which maturity evidence is missing?

Exercise: interpret a DMA pass

State exactly what the oracle established.

- No burst-boundary assertion failed. No other failure was reported.
- Give the two readings of this result.
- Name the additional oracle and its end-of-test check.

What to change on Monday

Make one existing claim auditable.

- Trace one expected-value dependency.
- Check whether one claimed behavior reaches an active oracle.
- Replace one vague milestone with observable evidence.

Scope and next reading

The foundations guide later technique choices.

- Read the Foundations handout and book Chapters 1–4.
- Next: verification planning, coverage and closure.
- This lesson omits tool setup, UVM implementation and formal algorithms.